AI Assisted Coding

LAB ASSIGNMENT – 6.3

Name: N.Trinandini
Batch: 32
Hall Ticket No: 2303A52071

Q1.

Task: Prompt AI to generate a function that displays all Automorphic numbers between 1 and 1000 using a for loop.

• Instructions:

o Get AI-generated code to list Automorphic numbers using a for loop.

o Analyze the correctness and efficiency of the generated logic.

o Ask AI to regenerate using a while loop and compare both implementations.

Expected Output #1:

• Correct implementation that lists Automorphic numbers using both loop types, with explanation.

PROMPT1:

```
Assignment_6.3.py > ...
#Write a python program to generate a function that displays all automorphic numbers between 1 to 1000 using loops
```

Code:

```python
import time
import tracemalloc

# Function to check if a number is automorphic
def is_automorphic(num):
    square = num * num
    return str(square).endswith(str(num))

# Function to display automorphic numbers
def display_automorphic_numbers(limit):
    automorphic_numbers = []
    for i in range(1, limit + 1):
        if is_automorphic(i):
            automorphic_numbers.append(i)
    return automorphic_numbers

# Start tracking memory
tracemalloc.start()

# Start time
start_time = time.time()

limit = 1000
automorphic_numbers = display_automorphic_numbers(limit)

print("Automorphic numbers between 1 and", limit, "are:", automorphic_numbers)

# End time
end_time = time.time()

# Get memory usage
current_memory, peak_memory = tracemalloc.get_traced_memory()

# Stop tracking memory
tracemalloc.stop()
```

```
# Stop tracking memory
tracemalloc.stop()

execution_time = end_time - start_time

print("Execution time using for loop: {:.6f} seconds".format(execution_time))
print("Current memory usage: {:.2f} KB".format(current_memory / 1024))
print("Peak memory usage: {:.2f} KB".format(peak_memory / 1024))
```

OUTPUT:

```
Automorphic numbers between 1 and 1000 are: [1, 5, 6, 25, 76, 376, 625]
Execution time using for loop: 0.001315 seconds
Current memory usage: 0.19 KB
Peak memory usage: 0.63 KB
```

PROMPT2:

```
#now write the same program using while loop
```

Code:

```
import time
import tracemalloc

# Function to check if a number is automorphic
def is_automorphic(num):
    return str(num * num).endswith(str(num))

# Optimized while-loop version
def display_automorphic_numbers_while(limit):
    automorphic_numbers = []
    i = 1
    while i <= limit:
        if is_automorphic(i):
            automorphic_numbers.append(i)
        i += 1
    return automorphic_numbers

# Start memory tracking
tracemalloc.start()

# Start time
start_time = time.time()

limit = 1000
automorphic_numbers_while = display_automorphic_numbers_while(limit)

print("Automorphic numbers between 1 and", limit, "using while loop are:", automorphic_numbers_while)

# End time
end_time = time.time()

# Memory usage
current_memory, peak_memory = tracemalloc.get_traced_memory()
tracemalloc.stop()
```

```
print("Automorphic numbers between 1 and", limit, "using while loop are:", automorphic_numbers_while)

# End time
end_time = time.time()

# Memory usage
current_memory, peak_memory = tracemalloc.get_traced_memory()
tracemalloc.stop()

execution_time_while = end_time - start_time

print("Execution time using while loop: {:.6f} seconds".format(execution_time_while))
print("Current memory usage: {:.2f} KB".format(current_memory / 1024))
print("Peak memory usage: {:.2f} KB".format(peak_memory / 1024))
```

EXPLANATION:

Generates and verifies a function that lists all Automorphic numbers between 1 and 1000 using a for loop and checks correctness of the logic.

Analyzes the efficiency and accuracy of the for-loop implementation.

Regenerates the solution using a while loop and compares both approaches with a brief explanation

Q2.

Task: Ask AI to write nested if-elif-else conditions to classify online shopping feedback as Positive, Neutral, or Negative based on a numerical rating (1–5).

• Instructions:

o Generate initial code using nested if-elif-else.

o Analyze correctness and readability.

o Ask AI to rewrite using dictionary-based or match-case structure.

Expected Output #2:

• Feedback classification function with explanation and an alternative approach.

Prompt1:

```
#2
#Write a python program to generate nested if-elif-else conditions to classify online shopping feedback as Positive, Neutral, or Negative based on a numerical rating (1-
```

CODE:

```python
Assignment_6.3.py > ...
import time
import tracemalloc

def classify_feedback(rating):
    if 4 <= rating <= 5:
        return "Positive"
    elif rating == 3:
        return "Neutral"
    elif 1 <= rating <= 2:
        return "Negative"
    else:
        return "Invalid rating. Please provide a rating between 1 and 5."

# Start memory tracking
tracemalloc.start()

# Start time
start_time = time.time()

ratings = [5, 4, 3, 2, 1, 6, 0]
for rating in ratings:
    classification = classify_feedback(rating)
    print(f"Rating: {rating} - Feedback: {classification}")

# End time
end_time = time.time()

# Get memory usage
current_memory, peak_memory = tracemalloc.get_traced_memory()
tracemalloc.stop()

execution_time = end_time - start_time

print("Execution time using if-elif-else: {:.6f} seconds".format(execution_time))
print("Current memory usage: {:.2f} KB".format(current_memory / 1024))
print("Peak memory usage: {:.2f} KB".format(peak_memory / 1024))
```

OUTPUT:

```
Rating: 5 - Feedback: Positive
Rating: 4 - Feedback: Positive
Rating: 3 - Feedback: Neutral
Rating: 2 - Feedback: Negative
Rating: 1 - Feedback: Negative
Rating: 6 - Feedback: Invalid rating. Please provide a rating between 1 and 5.
Rating: 0 - Feedback: Invalid rating. Please provide a rating between 1 and 5.
Execution time using if-elif-else: 0.027065 seconds
Current memory usage: 0.13 KB
Peak memory usage: 0.71 KB
PS D:\college\3-2\AIAC\AIAC LAB\28-01-2026>
```

PROMPT2: USING DICTIONARY

```
#now do it using dictionary-based
```

Code:

```python
import time
import tracemalloc
def classify_feedback_dict(rating):
    feedback_dict = {
        (4, 5): "Positive",
        (3, 3): "Neutral",
        (1, 2): "Negative"
    }
    for key in feedback_dict:
        if key[0] <= rating <= key[1]:
            return feedback_dict[key]
    return "Invalid rating. Please provide a rating between 1 and 5."

# Start memory tracking
tracemalloc.start()
# Start time
start_time = time.time()

# Example usage
ratings = [5, 4, 3, 2, 1, 6, 0]
for rating in ratings:
    classification = classify_feedback_dict(rating)
    print(f"Rating: {rating} - Feedback: {classification}")

# End time
end_time = time.time()

# Memory usage
current_memory, peak_memory = tracemalloc.get_traced_memory()
tracemalloc.stop()
execution_time = end_time - start_time

print("Execution time using dictionary-based approach: {:.6f} seconds".format(execution_time))
print("Current memory usage: {:.2f} KB".format(current_memory / 1024))
print("Peak memory usage: {:.2f} KB".format(peak_memory / 1024))
```

OUTPPUT:

```
Rating: 5 - Feedback: Positive
Rating: 4 - Feedback: Positive
Rating: 3 - Feedback: Neutral
Rating: 2 - Feedback: Negative
Rating: 1 - Feedback: Negative
Rating: 6 - Feedback: Invalid rating. Please provide a rating between 1 and 5.
Rating: 0 - Feedback: Invalid rating. Please provide a rating between 1 and 5.
Execution time using dictionary-based approach: 0.000973 seconds
Current memory usage: 0.13 KB
Peak memory usage: 0.71 KB
```

Prompt3: Using Match case

```
# Using match-case structure
```

Code:

```python
import time
import tracemalloc

def classify_feedback_match(rating):
    match rating:
        case 4 | 5:
            return "Positive"
        case 3:
            return "Neutral"
        case 1 | 2:
            return "Negative"
        case _:
            return "Invalid rating. Please provide a rating between 1 and 5."

# Start memory tracking
tracemalloc.start()

# Start time
start_time = time.time()

# Example usage
ratings = [5, 4, 3, 2, 1, 6, 0]
for rating in ratings:
    classification = classify_feedback_match(rating)
    print(f"Rating: {rating} - Feedback: {classification}")

# End time
end_time = time.time()

# Memory usage
current_memory, peak_memory = tracemalloc.get_traced_memory()
tracemalloc.stop()
execution_time = end_time - start_time

print("Execution time using match-case structure: {:.6f} seconds".format(execution_time))
print("Current memory usage: {:.2f} KB".format(current_memory / 1024))
print("Peak memory usage: {:.2f} KB".format(peak_memory / 1024))
```

Output:

```
Rating: 5 - Feedback: Positive
Rating: 4 - Feedback: Positive
Rating: 3 - Feedback: Neutral
Rating: 2 - Feedback: Negative
Rating: 1 - Feedback: Negative
Rating: 6 - Feedback: Invalid rating. Please provide a rating between 1 and 5.
Rating: 0 - Feedback: Invalid rating. Please provide a rating between 1 and 5.
Execution time using match-case structure: 0.001379 seconds
Current memory usage: 0.13 KB
Peak memory usage: 0.71 KB
```

Explanation:

Classifies online shopping feedback as Positive, Neutral, or Negative using nested if-elif-else based on ratings from 1 to 5.

Evaluates the initial solution for correctness and readability, ensuring clear conditions and valid rating handling.

Provides an alternative implementation using a dictionary-based approach or match-case structure for improved clarity and maintainability.

Q3.

Define a function named statistical_operations(tuple_num) that performs the following statistical operations on a tuple of numbers:

- **Minimum, Maximum**
- **Mean, Median, Mode**
- **Variance, Standard Deviation**

While writing the function, observe the code suggestions provided by GitHub Copilot.Make decisions to accept, reject, or modify the suggestions based on

their relevance and correctness

PROMPT:

```
#3
'''Write a python program to generate a function named statistical_operations(tuple_num) that performs the following statistical operations on a tuple of numbers:
Minimum, Maximum
Mean, Median, Mode
Variance, Standard Deviation'''
```

CODE:

```
import statistics as stats
def statistical_operations(tuple_num):
    minimum = min(tuple_num)
    maximum = max(tuple_num)
    mean = stats.mean(tuple_num)
    median = stats.median(tuple_num)
    try:
        mode = stats.mode(tuple_num)
    except stats.StatisticsError:
        mode = "No unique mode"
    variance = stats.variance(tuple_num)
    std_deviation = stats.stdev(tuple_num)
    return {
        "Minimum": minimum,
        "Maximum": maximum,
        "Mean": mean,
        "Median": median,
        "Mode": mode,
        "Variance": variance,
        "Standard Deviation": std_deviation
    }
# Example usage
data = (1, 2, 2, 3, 4, 5, 5, 5)
results = statistical_operations(data)
for operation, value in results.items():
    print(f"{operation}: {value}")
```

OUTPUT:

```
Minimum: 1
Maximum: 5
Mean: 3.375
Median: 3.5
Mode: 5
Variance: 2.5535714285714284
Standard Deviation: 1.5979898086569353
PS D:\college\3-2\AIAC\AIAC LAB\28-01-2026>
```

Explanation:

Defines a function statistical_operations(tuple_num) that computes minimum, maximum, mean, median, mode, variance, and standard deviation from a tuple of numbers.

Uses appropriate built-in/statistical logic to ensure correctness of each statistical measure.

Evaluates GitHub Copilot suggestions by accepting relevant ones, modifying partially correct ones, and rejecting incorrect or inefficient suggestions to improve code quality.

Q4.

Prompt: Create a class Teacher with attributes teacher_id, name, subject, and experience. Add a method to display teacher details.

- **Expected Output: Class with initializer, method, and object creation.**

PROMPT:

```
#4
#Write a python program to Create a class Teacher with attributes teacher_id, name, subject, and experience.Class with initializer, method, object creation and display
# teacher details.
```

CODE:

```
class Teacher:
    def __init__(self, teacher_id, name, subject, experience):
        self.teacher_id = teacher_id
        self.name = name
        self.subject = subject
        self.experience = experience
    def display_details(self):
        print(f"Teacher ID: {self.teacher_id}")
        print(f"Name: {self.name}")
        print(f"Subject: {self.subject}")
        print(f"Experience: {self.experience} years")
        print("----------------------------------------------------------------------------------------------------")
# Example usage
teacher1 = Teacher(101, "Alice Smith", "Mathematics", 10)
teacher1.display_details()
teacher2 = Teacher(102, "Bob Johnson", "Science", 8)
teacher2.display_details()
teacher3 = Teacher(103, "Charlie Brown", "History", 5)
teacher3.display_details()
teacher4 = Teacher(104, "Diana Prince", "English", 12)
teacher4.display_details()
teacher5 = Teacher(105, "Ethan Hunt", "Physical Education", 7)
teacher5.display_details()
teacher6 = Teacher(106, "Fiona Gallagher", "Art", 4)
teacher6.display_details()
```

OUTPUT:

```
Teacher ID: 101
Name: Alice Smith
Subject: Mathematics
Experience: 10 years
----------------------------------------------------------------------------------------------------
Teacher ID: 102
Name: Bob Johnson
Subject: Science
Experience: 8 years
----------------------------------------------------------------------------------------------------
Teacher ID: 103
Name: Charlie Brown
Subject: History
Experience: 5 years
----------------------------------------------------------------------------------------------------
Teacher ID: 104
Name: Diana Prince
Subject: English
Experience: 12 years
----------------------------------------------------------------------------------------------------
Teacher ID: 105
Name: Ethan Hunt
```

```
Experience: 5 years
------------------------------------------------------------------------------------------
Teacher ID: 104
Name: Diana Prince
Subject: English
Experience: 12 years
------------------------------------------------------------------------------------------
Teacher ID: 105
Name: Ethan Hunt
Subject: Physical Education
Experience: 7 years
------------------------------------------------------------------------------------------
Teacher ID: 106
Name: Fiona Gallagher
Subject: Art
Experience: 4 years
------------------------------------------------------------------------------------------
```

EXPLANATION:

Defines a Teacher class with attributes teacher_id, name, subject, and experience using an initializer method.

Includes a method to display teacher details in a clear and readable format.

Demonstrates object creation and method invocation to access and show teacher information.

Q5.

Use zero-shot prompting to instruct an AI tool to generate a function

that validates an Indian mobile number.

Requirements

• The function must ensure the mobile number:

o Starts with 6, 7, 8, or 9

o Contains exactly 10 digits

Expected Output

• A valid Python function that performs all required validations

without using any input-output examples in the prompt

PROMPT:

```
#5
#Write a python program to generate function that validates Indian phone numbers starting with 6, 7, 8, or 9 and having a total of 10 digits.
```

CODE:

```python
import re
def validate_indian_phone_number(phone_number):
    pattern = r'^[6-9]\d{9}$'
    if re.match(pattern, phone_number):
        return True
    else:
        return False
# Example usage
phone_numbers = ["9876543210", "1234567890", "8765432109", "5678901234", "9123456780"]
for number in phone_numbers:
    if validate_indian_phone_number(number):
        print(f"{number} is a valid Indian phone number.")
    else:
        print(f"{number} is not a valid Indian phone number.")
```

OUTPUT:

```
PS D:\college\3-2\AIAC\AIAC LAB\28-01-2026> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/28-01-2026/Assignment_6.3.py"
9876543210 is a valid Indian phone number.
1234567890 is not a valid Indian phone number.
8765432109 is a valid Indian phone number.
5678901234 is not a valid Indian phone number.
9123456780 is a valid Indian phone number.
```

EXPLANATON:

Validates that the mobile number contains exactly 10 digits and consists only of numeric characters.

Ensures the number starts with 6, 7, 8, or 9, as required for Indian mobile numbers.

Uses zero-shot prompting logic to perform all checks directly without relying on any input–output examples.

Q6.

Task: Write a function using AI that finds all Armstrong numbers in a user-

specified range (e.g., 1 to 1000).

Instructions:

- **Use a for loop and digit power logic.**
- **Validate correctness by checking known Armstrong numbers (153, 370,**

etc.).

- **Ask AI to regenerate an optimized version (using list comprehensions).**

Expected Output #7:

- **Python program listing Armstrong numbers in the range.**
- **Optimized version with explanation.**

PROMPT1:

```
#6
#Write a python code to generate function that finds all Armstrong numbers in a user- specified range (e.g., 1 to 1000).
```

CODE:

```
import time
import tracemalloc

def is_armstrong(num):
    num_str = str(num)
    num_digits = len(num_str)
    return sum(int(digit) ** num_digits for digit in num_str) == num

def find_armstrong_numbers(start, end):
    armstrong_numbers = []
    for i in range(start, end + 1):
        if is_armstrong(i):
            armstrong_numbers.append(i)
    return armstrong_numbers

# Start memory tracking
tracemalloc.start()

# Start time
start_time = time.time()

# Example usage
start_range = 1
end_range = 1000
armstrong_numbers = find_armstrong_numbers(start_range, end_range)

print(f"Armstrong numbers between {start_range} and {end_range} are: {armstrong_numbers}")

# End time
end_time = time.time()
```

```python
# Start memory tracking
tracemalloc.start()

# Start time
start_time = time.time()

# Example usage
start_range = 1
end_range = 1000
armstrong_numbers = find_armstrong_numbers(start_range, end_range)

print(f"Armstrong numbers between {start_range} and {end_range} are: {armstrong_numbers}")

# End time
end_time = time.time()

# Memory usage
current_memory, peak_memory = tracemalloc.get_traced_memory()
tracemalloc.stop()

execution_time = end_time - start_time

print("Execution time using loops: {:.6f} seconds".format(execution_time))
print("Current memory usage: {:.2f} KB".format(current_memory / 1024))
print("Peak memory usage: {:.2f} KB".format(peak_memory / 1024))
```

OUTPUT:

```
Armstrong numbers between 1 and 1000 are: [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]
Execution time using loops: 0.002289 seconds
Current memory usage: 0.25 KB
Peak memory usage: 0.86 KB
```

Prompt2:

```
#Now,write the same program using list comprehension.
```

Code:

```python
import time
import tracemalloc

def is_armstrong(num):
    num_str = str(num)
    num_digits = len(num_str)
    return sum(int(digit) ** num_digits for digit in num_str) == num

def find_armstrong_numbers_list_comp(start, end):
    return [i for i in range(start, end + 1) if is_armstrong(i)]

# Start memory tracking
tracemalloc.start()

# Start time
start_time = time.time()

# Example usage
start_range = 1
end_range = 1000
armstrong_numbers_list_comp = find_armstrong_numbers_list_comp(start_range, end_range)

print(f"Armstrong numbers between {start_range} and {end_range} using list comprehension are: {armstrong_numbers_list_comp}")

# End time
end_time = time.time()

# Memory usage
current_memory, peak_memory = tracemalloc.get_traced_memory()
tracemalloc.stop()

execution_time_list_comp = end_time - start_time

print("Execution time using list comprehension: {:.6f} seconds".format(execution_time_list_comp))
print("Current memory usage: {:.2f} KB".format(current_memory / 1024))
print("Peak memory usage: {:.2f} KB".format(peak_memory / 1024))
```

Output:

```
Armstrong numbers between 1 and 1000 using list comprehension are: [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]
Execution time using list comprehension: 0.002615 seconds
Current memory usage: 0.25 KB
Peak memory usage: 0.93 KB
```

EXPLANATION:

Finds all Armstrong numbers in the given range by using a loop to calculate the sum of each digit raised to the power of the number of digits (e.g., 153, 370, 371).

The optimized version uses list comprehension, reducing code length and slightly improving execution speed while maintaining correctness.

Q7 .Task: Generate a function using AI that displays all Happy Numbers within a user-specified range (e.g., 1 to 500).

Instructions:

- **Implement the logic using a loop: repeatedly replace a number with the sum of the squares of its digits until the result is either 1 (Happy Number) or enters a cycle (Not Happy).**
- **Validate correctness by checking known Happy Numbers (e.g., 1, 7, 10, 13, 19, 23, 28…).**
- **Ask AI to regenerate an optimized version (e.g., by using a set to detect cycles instead of infinite loops).**

Expected Output #8:

- **Python program that prints all Happy Numbers within a range.**
- **Optimized version using cycle detection with explanation.**

PROMPT1:

```
#7
#write a python program with user input to display happy numbers range(1to500)using a loop
```

CODE:

```
import time as time
def is_happy_number(num):
    seen = set()
    while num != 1 and num not in seen:
        seen.add(num)
        num = sum(int(digit) ** 2 for digit in str(num))
    return num == 1
def find_happy_numbers(limit):
    happy_numbers = []
    for i in range(1, limit + 1):
        if is_happy_number(i):
            happy_numbers.append(i)
    return happy_numbers
start_time = time.time()
limit = 500
happy_numbers = find_happy_numbers(limit)          (variable) happy_numbers: list
print("Happy numbers between 1 and", limit, "are:", happy_numbers)
end_time = time.time()
execution_time = end_time - start_time
print("Execution time using loop: {:.6f} seconds".format(execution_time))
```

OUTPUT:

```
Happy numbers between 1 and 500 are: [1, 7, 10, 13, 19, 23, 28, 31, 32, 44, 49, 68, 70, 79, 82, 86, 91, 94, 97, 100, 103, 109, 129, 130, 133, 139, 167, 176, 188, 190, 192, 193, 203, 208
, 219, 226, 230, 236, 239, 262, 263, 280, 291, 293, 301, 302, 310, 313, 319, 320, 326, 329, 331, 338, 356, 362, 365, 367, 368, 376, 379, 383, 386, 391, 392, 397, 404, 409, 440, 446, 464
, 469, 478, 487, 490, 496]
Execution time using loop: 0.003015 seconds
```

Prompt2:

```
#Now, give me more optimised code for happy numbers using list comprehension with cycle detection
```

Code:

```python
import time as time
def is_happy_number(num):
    seen = set()
    while num != 1 and num not in seen:
        seen.add(num)
        num = sum(int(digit) ** 2 for digit in str(num))
    return num == 1
def find_happy_numbers_optimized(limit):
    return [i for i in range(1, limit + 1) if is_happy_number(i)]
start_time = time.time()
limit = 500
happy_numbers_optimized = find_happy_numbers_optimized(limit)
print("Happy numbers between 1 and", limit, "using optimized approach are:", happy_numbers
_optimized)
end_time = time.time()
execution_time_optimized = end_time - start_time
print("Execution time using optimized approach: {:.6f} seconds".format(execution_time_optimized))
```

Output:

```
Happy numbers between 1 and 500 using optimized approach are: [1, 7, 10, 13, 19, 23, 28, 31, 32, 44, 49, 68, 70, 79, 82, 86, 91, 94, 97, 100, 103, 109, 129, 130, 133, 139, 167, 176, 188
, 190, 192, 193, 203, 208, 219, 226, 230, 236, 239, 262, 263, 280, 291, 293, 301, 302, 310, 313, 319, 320, 326, 329, 331, 338, 356, 362, 365, 367, 368, 376, 379, 383, 386, 391, 392, 397
, 404, 409, 440, 446, 464, 469, 478, 487, 490, 496]
Execution time using optimized approach: 0.003272 seconds
```

EXPLANATION:

Displays all Happy Numbers in the given range by repeatedly replacing a number with the sum of the squares of its digits and checking if it reaches 1 (e.g., 1, 7, 10, 13, 19).

The optimized approach uses a set for cycle detection, preventing infinite loops and improving correctness and efficiency.

Q8]

Task: Generate a function using AI that displays all Strong Numbers (sum of factorial of digits equals the number, e.g., 145 = 1!+4!+5!) within a given range.

Instructions:

- **Use loops to extract digits and calculate factorials.**
- **Validate with examples (1, 2, 145).**
- **Ask AI to regenerate an optimized version (precompute digit factorials).**

Expected Output #9:

- **Python program that lists Strong Numbers.**
- **Optimized version with explanation.**

PROMPT1:

```
#8
#Write a python program to generate a function that displays strong numbers using loops
```

CODE:

```python
import time as time
def is_strong_number(num):
    def factorial(n):
        if n == 0 or n == 1:
            return 1
        else:
            return n * factorial(n - 1)
    sum_of_factorials = sum(factorial(int(digit)) for digit in str(num))
    return sum_of_factorials == num
def find_strong_numbers(limit):
    strong_numbers = []
    for i in range(1, limit + 1):
        if is_strong_number(i):
            strong_numbers.append(i)
    return strong_numbers
start_time = time.time()
limit = 500
strong_numbers = find_strong_numbers(limit)
print("Strong numbers between 1 and", limit, "are:", strong_numbers)
end_time = time.time()
execution_time = end_time - start_time
print("Execution time using loop: {:.6f} seconds".format(execution_time))
```

OUTPUT:

```
Strong numbers between 1 and 500 are: [1, 2, 145]
Execution time using loop: 0.000813 seconds
PS D:\college\3-2\AIAC\AIAC LAB\28-01-2026>
```

Prompt2:

```
#Now,write the same program in optimised way in precompute digit factorials
```

Code:

```python
import time as time
def is_strong_number_optimized(num, factorials):
    sum_of_factorials = sum(factorials[int(digit)] for digit in str(num))
    return sum_of_factorials == num
def find_strong_numbers_optimized(limit):
    factorials = {i: factorial(i) for i in range(10)}
    strong_numbers = [i for i in range(1, limit + 1) if is_strong_number_optimized(i, factorials)]
    return strong_numbers
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
start_time = time.time()
limit = 500
strong_numbers_optimized = find_strong_numbers_optimized(limit)
print("Strong numbers between 1 and", limit, "using optimized approach are:", strong_numbers_optimized)
end_time = time.time()
execution_time_optimized = end_time - start_time
print("Execution time using optimized approach: {:.6f} seconds".format(execution_time_optimized))
```

Output:

```
Strong numbers between 1 and 500 using optimized approach are: [1, 2, 145]
Execution time using optimized approach: 0.000451 seconds
PS D:\college\3-2\AIAC\AIAC LAB\28-01-2026>
```

EXPLANATION:

The program identifies Strong Numbers by checking whether the sum of factorials of digits equals the number (e.g., 1, 2, 145).

The optimized version precomputes digit factorials (0–9), avoiding repeated factorial calculations and improving efficiency.

Q9.

Objective

Use few-shot prompting (2–3 examples) to instruct the AI to create a function that parses a nested dictionary representing student information.

Requirements

• The function should extract and return:

o Full Name

o Branch

o SGPA

Expected Output

A reusable Python function that correctly navigates and extracts values from nested dictionaries based on the provided examples

PROMPT:

```
#9
'''Write a python program to generate a fuction representing students information
The function should allow adding new students, updating existing student details, and displaying all student information.
Take the input from the user for student details
Input1: student_name: N.Trinandini
        Branch: CSE-AI&ML
        CGPA: 9.1
Input2: student_name: T.Sai Sathwil
        Branch: CSE-
        CGPA: 8.9
Input3: student_name: Ch.Karthikeya
        Branch: CSE
        CGPA: 9.3
Output: Display all student information in a structured format of nested dictinoary.'''
```

CODE:

```python
class StudentDatabase:
    def __init__(self):
        self.students = {}
    def add_student(self, student_id, name, branch, cgpa):
        self.students[student_id] = {
            "Name": name,
            "Branch": branch,
            "CGPA": cgpa
        }
    def update_student(self, student_id, name=None, branch=None, cgpa=None):
        if student_id in self.students:
            if name:
                self.students[student_id]["Name"] = name
            if branch:
                self.students[student_id]["Branch"] = branch
            if cgpa:
                self.students[student_id]["CGPA"] = cgpa
        else:
            print("Student ID not found.")
    def display_students(self):
        for student_id, details in self.students.items():
            print(f"Student ID: {student_id}")
            for key, value in details.items():
                print(f"  {key}: {value}")
            print("--------------------------------------------------")
# Example usage
student_db = StudentDatabase()
student_db.add_student(1, "N. Trinandini", "CSE-AI&ML", 9.1)
student_db.add_student(2, "T. Sai Sathwil", "CSE-",
8.9)
student_db.add_student(3, "Ch. Karthikeya", "CSE", 9.3)
student_db.display_students()
```

OUTPUT:

```
Student ID: 1
  Name: N. Trinandini
  Branch: CSE-AI&ML
  CGPA: 9.1
--------------------------------------------------
Student ID: 2
  Name: T. Sai Sathwil
  Branch: CSE-
  CGPA: 8.9
--------------------------------------------------
Student ID: 3
  Name: Ch. Karthikeya
  Branch: CSE
  CGPA: 9.3
--------------------------------------------------
```

EXPLANATION:

Navigates nested dictionaries to extract personal and academic details accurately.

Reusable function works for any student dictionary following the same structure.

Time Complexity: O(1) since direct key access is used.

Q10.

Task: Generate a function using AI that displays all Perfect Numbers within a user-specified range (e.g., 1 to 1000).

Instructions:

• A Perfect Number is a positive integer equal to the sum of its proper divisors (excluding itself).

o Example: 6 = 1 + 2 + 3, 28 = 1 + 2 + 4 + 7 + 14.

• Use a for loop to find divisors of each number in the range.

• Validate correctness with known Perfect Numbers (6, 28, 496…).

• Ask AI to regenerate an optimized version (using divisor check only up to √n).

PROMPT:

```
#10
#Write a python program  to generate a function that displays perfect numbers using loops
```

CODE:

```python
import time as time
def is_perfect_number(num):
    sum_of_divisors = sum(i for i in range(1, num) if num % i == 0)
    return sum_of_divisors == num

def find_perfect_numbers(limit):
    perfect_numbers = []
    for i in range(1, limit + 1):
        if is_perfect_number(i):
            perfect_numbers.append(i)
    return perfect_numbers
start_time = time.time()
limit = 500
perfect_numbers = find_perfect_numbers(limit)
print("Perfect numbers between 1 and", limit, "are:", perfect_numbers)
end_time = time.time()
execution_time = end_time - start_time
print("Execution time using loop: {:.6f} seconds".format(execution_time))
```

OUTPUT:

```
Perfect numbers between 1 and 500 are: [6, 28, 496]
Execution time using loop: 0.004182 seconds
PS D:\college\3-2\AIAC\AIAC LAB\28-01-2026>
```

Prompt 2:

```
#Now,write the same program in optimised way using divisor check only up to sqrt(n)
```

Code:

```python
import time as time
import math
def is_perfect_number_optimized(num):
    if num < 2:
        return False
    sum_of_divisors = 1
    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            sum_of_divisors += i
            if i != num // i:
                sum_of_divisors += num // i
    return sum_of_divisors == num
def find_perfect_numbers_optimized(limit):
    perfect_numbers = [i for i in range(1, limit + 1) if is_perfect_number_optimized(i)]
    return perfect_numbers
start_time = time.time()
limit = 500
perfect_numbers_optimized = find_perfect_numbers_optimized(limit)
print("Perfect numbers between 1 and", limit, "using optimized approach are:", perfect_numbers_optimized)
end_time = time.time()
execution_time_optimized = end_time - start_time
print("Execution time using optimized approach: {:.6f} seconds".format(execution_time_optimized))
```

Output:

```
Perfect numbers between 1 and 500 using optimized approach are: [6, 28, 496]
Execution time using optimized approach: 0.000577 seconds
PS D:\college\3-2\AIAC\AIAC LAB\28-01-2026>
```

EXPLANATION:

Lists all Perfect Numbers in the given range by checking if the sum of proper divisors equals the number (e.g., 6, 28, 496).

Uses an optimized divisor check only up to √n, reducing unnecessary iterations while maintaining correctness.

Time Complexity: Normal approach $O(n^2)$, Optimized approach $O(n\sqrt{n})$.